

PYNQ 프레임워크의 하드웨어 추상화 메커니즘 분석: XSA/HWH 파싱부터 동적 MMIO 라이브러리 생성까지

서론

PYNQ(Python Productivity for Zynq) 프레임워크는 AMD Zynq SoC(System-on-Chip) 플랫폼에서 프로그래머블 로직(PL)의 성능을 고수준 언어인 파이썬으로 손쉽게 활용할 수 있도록 설계된 오픈소스 프로젝트이다.¹ 전통적인 FPGA 개발이 VHDL이나 Verilog와 같은 하드웨어 기술 언어(HDL)에 대한 깊은 전문성을 요구했던 것과 달리, PYNQ는 소프트웨어 개발자, 연구자, 시스템 설계자들이 복잡한 하드웨어 설계 도구 없이도 FPGA의 병렬 처리 능력과 하드웨어 가속의 이점을 누릴 수 있도록 진입 장벽을 낮추는 것을 목표로 한다.¹ 이러한 목표를 달성하기 위한 PYNQ의 핵심 철학은 하드웨어 설계를 '오버레이(Overlay)'라는 소프트웨어 라이브러리처럼 취급하여 동적으로 로드하고 제어하는 것이다.³

본 보고서는 PYNQ 프레임워크가 이러한 동적 하드웨어 제어를 구현하는 내부 메커니즘을 심층적으로 분석한다. 특히, 하드웨어 설계 도구인 Vivado에서 생성된 하드웨어 명세 파일(XSA 또는 HWH)을 파싱하여, 해당 하드웨어에 포함된 IP(Intellectual Property) 코어들을 자동으로 인식하고, 이를 위한 파이썬 기반의 메모리 맵 입출력(MMIO) 라이브러리를 동적으로 생성하는 전 과정을 상세히 추적한다. 이 과정에서 PYNQ는 리눅스 커널 수준의 저수준 인터페이스를 추상화하고, DefaultIP라는 범용 드라이버를 제공하며, 나아가 사용자가 특정 IP에 최적화된 고수준 드라이버를 손쉽게 개발하고 바인딩할 수 있는 정교한 구조를 제공한다. 본 보고서는 이러한 각 단계의 기술적 세부 사항과 아키텍처 설계를 소스 코드 수준에서 분석하여 PYNQ 프레임워크의 핵심 동작 원리에 대한 포괄적인 이해를 제공하고자 한다.

제 1장: 하드웨어 핸드오프 - PL 설계를 XSA 및 HWH 파일로 캡슐화

PYNQ 소프트웨어 스택으로의 입력은 하드웨어 메타데이터이며, 이 메타데이터의 기원과 내용을 이해하는 것은 전체 프로세스를 파악하는 첫걸음이다. 이 장에서는 Vivado의 그래픽 기반 하드웨어 설계가 어떻게 기계가 읽을 수 있는 형식으로 변환되는지 상세히 설명한다.

Vivado IP 통합기의 역할

모든 프로세스는 AMD Vivado 설계 도구의 IP 통합기(IP Integrator) 환경에서 시작된다.⁴ 설계자는 이곳에서 Zynq Processing System (PS) 블록을 중심으로 AXI GPIO, DMA 컨트롤러, 또는 HLS(High-Level Synthesis)로 생성된 커스텀 연산 가속기와 같은 다양한 IP 코어를 인스턴스화하고, AXI 버스를 통해 이들을 상호 연결한다. 이 그래픽 기반의 블록 다이어그램은 구현될 하드웨어 시스템의 토폴로지, 즉 구성 요소와 그들 간의 연결 관계에 대한 모든 정보를 담고 있는 원천(source of truth)이다.

하드웨어 핸드오프 파일 (HWH): 표준화된 청사진

블록 다이어그램 설계가 완료되고 출력물(Output Products) 생성 과정이 실행되면, Vivado는 HWH(.hwh) 파일을 생성한다.⁵ 이 파일은 하드웨어 시스템에 대한 포괄적인 XML 기반의 기술서로, 하드웨어와 소프트웨어 영역 간의 표준화된 '데이터시트' 또는 '계약서' 역할을 수행한다.⁷ HWH 파일은 다음과 같은 핵심 정보를 포함한다:

- **IP 인벤토리:** 설계에 포함된 모든 IP 인스턴스의 목록과 각 IP를 고유하게 식별하는 VLNV(Vendor:Library:Name:Version) 식별자.
- **메모리 맵:** 시스템의 전체 주소 맵 정보. 각 메모리 맵(주로 AXI-Lite 인터페이스) 주변장치의 물리적 기준 주소(phys_addr)와 주소 범위(addr_range)를 명시한다.⁴
- **인터럽트 연결:** 어떤 IP의 인터럽트 출력 신호가 어떤 AXI 인터럽트 컨트롤러의 몇 번 라인에 연결되었는지에 대한 정보.⁵
- **GPIO 연결:** PS에서 PL로 연결되는 GPIO 신호의 연결 정보.
- **클럭 설정:** PL 영역에서 사용되는 시스템 클럭의 구성 정보.⁶

Xilinx 서포트 아카이브 (XSA): 포괄적인 컨테이너

XSA(.xsa) 파일은 최신 Vivado 버전에서 사용하는 표준 컨테이너 포맷으로, 본질적으로는 ZIP 아카이브 파일이다.⁵ 이 파일은 소프트웨어 개발에 필요한 모든 구성 요소를 하나의 패키지로

묶어 관리의 편의성을 높인다.¹¹ XSA 파일은 일반적으로 다음을 포함한다:

- PL을 프로그래밍하기 위한 비트스트림(.bit) 파일
- 상술한 HWH(.hwh) 파일
- Vitis나 PetaLinux와 같은 소프트웨어 도구가 시스템을 초기화하는 데 필요한 파일들 (psu_init.c, .tcl 등)⁵

PYNQ 프레임워크는 초기에 .bit 파일과 .hwh 파일을 쌍으로 사용하는 방식으로 설계되었다.⁶ 하지만 PYNQ v3.0 이상부터는 XSA 포맷을 직접 지원하도록 발전하여, 단일 파일만으로 오버레이를 배포하고 로드할 수 있게 되어 사용자 편의성이 크게 향상되었다.¹³

이 HWH/XSA 파일의 생성은 단순한 파일 출력을 넘어, 하드웨어와 소프트웨어 통합 방식의 근본적인 패러다임 전환을 의미한다. 전통적인 Zynq 임베디드 C 개발 환경에서는 Vitis가 xparameters.h와 같은 헤더 파일을 생성하여 각 IP의 기준 주소를 #define 매크로로 정의했다.⁴ 이는 정적인 컴파일 타임에 주소 정보가 결정되는 방식이다. 반면, PYNQ는 런타임에 오버레이를 동적으로 교체하는 것을 목표로 하므로¹, 이러한 정적 방식은 적합하지 않다. HWH 파일은

xparameters.h가 제공하는 정보(그리고 그 이상)를 구조화된 XML 형식으로 제공함으로써 이 문제를 해결한다. 따라서 HWH 파일은 PYNQ의 동적인 특성을 가능하게 하는 핵심 요소이다. 이 파일을 통해 파이썬 환경은 정적으로 컴파일된 보드 지원 패키지(BSP) 없이도 런타임에 하드웨어 토폴로지를 '발견(discover)'할 수 있으며, 이것이 바로 "Zynq를 위한 파이썬 생산성"이라는 PYNQ의 철학을 구현하는 기술적 기반이 된다.

제 2장: PYNQ 파싱 엔진 - 파일에서 인메모리 메타데이터로

하드웨어 설계가 HWH/XSA 파일로 캡슐화되면, 다음 단계는 PYNQ 소프트웨어 스택이 이 파일을 읽고 해석하여 프레임워크가 활용할 수 있는 구조화된 인메모리 데이터로 변환하는 것이다. 이 장에서는 이 변환 과정을 담당하는 PYNQ의 파싱 엔진 메커니즘을 해부한다.

PYNQ-Metadata 라이브러리 소개

최신 PYNQ 버전에서 모든 하드웨어 메타데이터 파싱은 PYNQ-Metadata라는 독립적인 파이썬 라이브러리에 의해 수행된다.¹⁶ 이 라이브러리의 존재는 PYNQ 아키텍처가 성숙하고 모듈화되었음을 보여주는 중요한 증거이다.

PYNQ-Metadata는 다음과 같은 명확한 관심사 분리(Separation of Concerns) 디자인 패턴을

따른다.

- **pynqmetadata/frontends**: XSA, HWH, JSON 등 다양한 입력 파일 형식을 처리하는 파서 모듈들을 포함한다. 이 디렉토리는 파일 핸들링과 관련된 복잡한 세부 사항을 캡슐화하여 격리시킨다.¹⁶
- **pynqmetadata/models**: 하드웨어 시스템을 파일 형식에 구애받지 않는 추상적인 객체 모델로 표현하기 위한 파이썬 클래스들을 정의한다 (**Module**, **Block**, **Port**, **Register** 등). PYNQ의 나머지 부분은 이 안정적인 내부 모델과 상호작용한다.¹⁶

XSA 및 HWH 파일 파싱 과정

사용자가 파이썬 코드에서 `Overlay('design.xsa')`를 호출하면, PYNQ-Metadata 라이브러리가 내부적으로 활성화되어 다음과 같은 과정을 거친다.¹⁴

1. XSA 파일 처리 (**XsaFrontend**):

- **XsaFrontend**는 입력된 **.xsa** 파일을 ZIP 아카이브로 취급한다.
- 파이썬의 **zipfile** 라이브러리를 사용하여 아카이브 내부를 탐색하고, **.hwh** 확장자를 가진 첫 번째 파일을 찾는다.¹⁰
- HWH 파일을 찾으면 그 내용을 메모리로 추출한 뒤, 실제 XML 파싱 작업은 **HwhFrontend**에 위임한다. 이는 전형적인 위임(**delegation**) 패턴이다.

2. HWH 파일 처리 (**HwhFrontend**):

- **HwhFrontend**는 HWH 파일의 XML 텍스트 내용을 입력으로 받는다.
- 파이썬 표준 라이브러리인 **xml.etree.ElementTree**를 사용하여 XML 트리를 순회한다.
- XML 구조를 탐색하며 모듈(IP 블록), 포트, 메모리 인터페이스(MEMMAP), 파라미터 및 이들의 상호 연결 정보를 체계적으로 추출한다.
- 추출된 정보는 **pynqmetadata/models**에 정의된 추상 객체 모델(예: **Module**, **Block** 인스턴스)을 생성하고 속성을 채우는 데 사용된다.

최종 결과물: **RuntimeMetadataParser**

파싱이 완료되어 추상적인 하드웨어 모델이 메모리에 생성되면, 이 모델은 **RuntimeMetadataParser** 클래스로 전달된다. 이 클래스는 **Overlay** 클래스가 더 편리하게 사용할 수 있도록 여러 '뷰(view)' 또는 딕셔너리를 생성하는 역할을 한다. 대표적인 뷰는 다음과 같다¹⁷:

- **ip_dict**: PS에서 주소 지정이 가능한 모든 IP의 딕셔너리.
- **hierarchy_dict**: 설계 내의 계층 구조를 나타내는 딕셔너리.
- **mem_dict**: 설계의 모든 메모리 영역에 대한 딕셔너리.

- `interrupt_controllers`: 시스템의 모든 AXI 인터럽트 컨트롤러 디렉터리.

이러한 모듈식 아키텍처는 PYNQ 프레임워크의 핵심 로직을 AMD-Xilinx의 하드웨어 기술 파일 형식의 구체적인 사양으로부터 분리시키는 중요한 역할을 한다. AMD-Xilinx의 개발 도구와 출력 파일 형식은 시간이 지남에 따라 진화한다 (예: 과거의 HDF에서 현재의 XSA로의 변화).⁷ 만약 파싱 로직이 메인

`pynq` 라이브러리에 하드코딩되어 있었다면, 도구의 버전이 업데이트될 때마다 PYNQ 프레임워크 전체가 영향을 받아 유지보수가 매우 어려워졌을 것이다. 별도의 PYNQ-Metadata 패키지를 만듦으로써, PYNQ 개발팀은 이러한 불안정한 의존성을 성공적으로 격리했다. 향후 Vivado가 HWH 파일의 XML 스키마를 변경하더라도, PYNQ-Metadata의 `HwhFrontend`만 수정하면 된다. 핵심적인 `pynq.overlay` 로직은 안정적인 추상 `models`와만 상호작용하기 때문에 변경될 필요가 없다. 이는 장기적인 유지보수성과 생태계 확장을 촉진하는 전략적인 아키텍처 설계의 결과물이다.

제 3장: 커널-사용자 공간의 간극 연결 - `pynq.mmio.MMIO` 추상화

하드웨어 메타데이터가 파싱되어 메모리에 구조화되면, 다음 과제는 사용자 공간(`user-space`)에서 실행되는 파이썬 프로세스가 커널의 보호를 받는 물리 메모리 주소에 직접 접근하여 하드웨어를 제어하는 것이다. 이 장에서는 `pynq.mmio.MMIO` 클래스의 소스 코드 수준 분석을 통해 PYNQ가 운영 체제의 제약을 우회하고 이 문제를 어떻게 해결하는지 탐구한다.

근본적인 과제와 해결책: `/dev/mem`과 `mmap`

리눅스와 같이 메모리 관리 장치(MMU)를 사용하는 현대 운영 체제에서, 사용자 애플리케이션은 가상 주소 공간에서 동작한다. 이는 안정성과 보안을 위해 애플리케이션이 시스템의 물리 메모리 주소에 직접 접근하는 것을 원천적으로 차단한다. PYNQ는 이 제약을 극복하기 위해 리눅스가 제공하는 표준 메커니즘을 활용한다.

- `os.open('/dev/mem',...)`: MMIO 클래스의 생성자(`__init__`)는 `/dev/mem`이라는 특수 문자 디바이스 파일을 연다.¹⁹ 이 파일은 리눅스 커널이 제공하는 인터페이스로, 시스템의 전체 물리 메모리를 하나의 연속된 파일처럼 보이게 한다. 이 파일에 접근하기 위해서는 루트(`root`) 권한이 필요하며, 이것이 소스 코드에 `os.geteuid() != 0`을 통해 권한을 확인하는 로직이 포함된 이유이다.¹⁹

- **mmap.mmap(...)**: 핵심적인 작업은 **mmap** 시스템 콜을 통해 이루어진다. **mmap**은 커널에게 특정 파일의 일부(이 경우 `/dev/mem`으로 대표되는 물리 메모리)를 호출한 프로세스의 가상 주소 공간에 직접 매핑하도록 지시한다.¹⁹
mmap 함수의 **offset** 인자에는 제어하고자 하는 IP 코어의 물리적 기준 주소(**base_addr**)가 전달된다. 이 과정을 통해, 파이썬 프로세스는 특정 가상 주소에 값을 읽고 쓰는 것만으로 실제 하드웨어의 물리 주소에 접근할 수 있게 된다.

페이지 정렬 처리

mmap 시스템 콜은 시스템의 페이지 크기(일반적으로 **4KB**)에 정렬된 메모리 단위로만 동작할 수 있다. 하지만 **Vivado**가 할당하는 IP의 기준 주소는 페이지 경계에 정렬되어 있다는 보장이 없다. **MMIO** 클래스는 이 문제를 내부적으로 처리한다.

1. 먼저, 제어하려는 물리 주소(**base_addr**)를 포함하는 페이지의 시작 주소를 계산한다:
`virt_base = base_addr & ~(mmap.PAGESIZE - 1)`.¹⁹
2. 다음으로, 이 페이지 시작 주소로부터 실제 IP의 기준 주소까지의 오프셋을 계산한다:
`virt_offset = base_addr - self.virt_base`.¹⁹
3. **mmap**을 호출할 때는 **virt_base**를 기준으로 페이지 전체(또는 여러 페이지)를 매핑하고, 이후 모든 접근 시에는 내부적으로 **virt_offset**을 더하여 정확한 물리 주소를 가리키도록 한다.

NumPy를 이용한 고성능 접근

메모리가 매핑된 후, **PYNQ**는 이 매핑된 메모리 버퍼 위에 **NumPy** 배열 뷰(**view**)를 생성한다: `self.array = np.frombuffer(...)`.²⁰ 이는 매우 효율적인 접근 방식이다. 이 **NumPy** 배열의 특정 인덱스에 값을 읽거나 쓰는 작업은 내부적으로 최적화된 **C** 코드를 통해 직접적인 메모리 로드/스토어 명령으로 변환된다. 이는 파이썬 인터프리터의 오버헤드를 피할 수 있어, 순수 파이썬으로 반복적인 메모리 접근을 수행하는 것보다 훨씬 빠른 성능을 제공한다.²¹

read()와 **write()** API 메소드들은 사용자가 제공한 **offset**을 기반으로 이 **NumPy** 배열의 올바른 인덱스를 계산하여 읽기 또는 쓰기 작업을 수행하는 간단한 인터페이스를 제공한다.²⁰

MMIO 클래스는 운영 체제 수준의 상당한 복잡성과 위험성을 안전하고 파이썬다운 **API** 뒤에 숨기는 훌륭한 추상화 계층이다. `/dev/mem`을 직접 사용하는 것은 강력하지만, 잘못된 주소에 값을 쓰면 시스템 전체가 멈출 수 있는 위험이 따른다. **MMIO** 클래스는 생성 시 지정된 **length** 범위 내에서만 접근을 허용함으로써 이러한 위험을 완화한다. 또한, 이 클래스의 구현 방식

변화는 플랫폼 이식성을 향한 전략적 전환을 보여준다. 초기 버전의 소스 코드 ¹⁹는

`/dev/mem`과 `mmap`을 직접 사용하는 구현을 보여주며, 이는 MMIO 로직을 Zynq-on-Linux 환경에 강하게 결합시킨다. 반면, 후기 버전의 소스 코드 ²⁴에서는

`Device`라는 추상화 계층(`device = Device.active_device`)이 도입되었다. 생성자는 이제 디바이스의 능력('MEMORY_MAPPED' 또는 'REGISTER_RW')을 확인하고 실제 메모리 매핑이나 레지스터 접근을 이 디바이스 객체에 위임한다. 이러한 아키텍처 개선은 PYNQ가 다른 플랫폼을 지원하는 데 결정적인 역할을 한다. 예를 들어, Alveo와 같은 PCIe 가속기 카드에서는 MMIO가 `/dev/mem`이 아닌 XRT(Xilinx Runtime) 드라이버 스택을 통해 처리된다. MMIO 클래스의 API는 사용자에게 동일하게 유지되지만, 백엔드 구현은 런타임에 감지된 하드웨어에 따라 선택된다. 이로 인해 MMIO 위에 구축된 모든 드라이버는 본질적으로 더 높은 이식성을 갖게 된다.

제 4장: 동적 인스턴스화 - Overlay 클래스와 DefaultIP 드라이버

PYNQ가 런타임에 하드웨어의 파이썬 객체 모델을 동적으로 생성하는 과정은 마치 '마법'처럼 보일 수 있다. 이 장에서는 Overlay 클래스의 초기화 과정과 모든 IP에 대한 보편적인 대체 드라이버(fallback driver)로서 DefaultIP가 수행하는 핵심적인 역할을 분석하여 그 '마법'의 원리를 밝힌다.

Overlay 클래스 초기화 과정

사용자가 주피터 노트북 등에서 `ol = Overlay("design.xsa")`와 같은 코드를 실행하면, 다음과 같은 다단계 프로세스가 진행된다 ²⁵:

1. 비트스트림 다운로드: `download=False` 옵션이 지정되지 않은 경우, XSA 파일에 포함된 비트스트림이 PL에 다운로드된다.
2. 메타데이터 파싱: 제 2장에서 설명한 PYNQ-Metadata 엔진을 사용하여 XSA/HWH 파일을 파싱한다.
3. `ip_dict` 생성: 파싱 결과로 `ip_dict`라는 딕셔너리가 생성된다. 이 딕셔너리의 키(key)는 Vivado 블록 다이어그램에 정의된 IP의 이름(예: `axi_gpio_0`)이며, 값(value)은 해당 IP의 메타데이터(`phys_addr`, `addr_range`, `type` 등)를 담고 있는 또 다른 딕셔너리다.⁸

_assign_drivers 로직: 동적 드라이버 할당의 핵심

`_assign_drivers`라는 내부 함수는 동적 인스턴스화 과정의 심장부이다.⁸ 이 함수는

`ip_dict`에 있는 모든 IP 항목을 순회하며 다음 작업을 수행한다:

- 각 IP에 대해 VLNV 타입 문자열(예: 'xilinx.com:ip:axi_gpio:2.0')을 추출한다.
- 이 타입 문자열이 PYNQ 프레임워크에 사전 등록된 특정 드라이버들의 전역 딕셔너리(`_ip_drivers`)에 존재하는지 확인한다.
- 일치하는 드라이버가 있는 경우: 해당 IP의 메타데이터 딕셔너리에 특정 드라이버 클래스(예: `pynq.lib.axigpio.AxiGPIO`)를 'driver' 키의 값으로 할당한다.
- 일치하는 드라이버가 없는 경우: `pynq.overlay.DefaultIP` 클래스를 기본 드라이버로 할당한다.⁸

동적 속성 생성과 **DefaultIP**의 역할

드라이버 할당이 완료되면, **Overlay** 클래스는 `ip_dict`의 각 IP 이름에 대해 동적으로 속성(attribute)을 생성한다. 사용자가 코드에서 `ol.my_axi_gpio_0`과 같이 해당 속성에 처음 접근할 때, 비로소 할당된 드라이버 클래스(**AxiGPIO** 또는 **DefaultIP**)의 인스턴스가 생성된다.

DefaultIP 클래스는 특정 드라이버가 작성되지 않은 모든 메모리 맵 IP를 위한 범용 드라이버이다.⁹

- **DefaultIP**의 생성자(`__init__`)는 자신이 인스턴스화되는 특정 IP의 메타데이터 딕셔너리를 인자로 받는다.
- 이 딕셔너리로부터 `phys_addr`와 `addr_range` 값을 사용하여 제 3장에서 설명한 `pynq.mmio.MMIO` 클래스의 내부 인스턴스를 생성한다.
- 이 과정을 통해, **DefaultIP**로 인스턴스화된 객체는 즉시 `add_ip.read(offset)` 및 `add_ip.write(offset, data)`와 같은 저수준 레지스터 접근 메소드를 제공할 수 있게 된다.²⁸

메타데이터 파싱과 **DefaultIP** 드라이버의 조합은 하드웨어 설계에 포함된 모든 메모리 맵 IP가 추가적인 소프트웨어 개발 노력 없이도 오버레이 로드 즉시 파이썬에서 접근 가능하도록 보장한다. 이는 하드웨어/소프트웨어 공동 설계 및 디버그 주기를 극적으로 가속화한다. 전통적인 워크플로우에서는 하드웨어 엔지니어가 IP를 추가하면, 소프트웨어 엔지니어는 새로운 **BSP**가 생성되기를 기다리고, 데이터시트를 참조하여 레지스터 오프셋을 파악한 후, 이를 제어하기 위한 **C** 코드를 작성해야 했다. 이는 느리고 오류가 발생하기 쉬운 과정이다. 반면, **PYNQ** 워크플로우에서는 하드웨어 엔지니어가 IP를 추가하고 새로운 `.xsa` 파일만 제공하면 된다. 소프트웨어 엔지니어는 이 파일을 **Overlay**로 로드하고, 새로 추가된 IP는 **DefaultIP** 드라이버와 함께 즉시 객체 속성으로 나타난다. `ol.new_ip_name.read(0x00)`와 같은

코드를 주피터 노트북에서 바로 실행하여 새로운 하드웨어의 상태 레지스터를 확인하고 검증할 수 있다. 이처럼 즉각적이고 상호작용적인 하드웨어 검증 능력은 PYNQ 철학의 핵심이며 ¹,

DefaultIP 클래스는 이러한 신속한 프로토타이핑(rapid-prototyping)을 가능하게 하는 결정적인 기술 요소이다.

제 5장: 고수준 추상화 - 커스텀 IP 드라이버 제작 및 바인딩

DefaultIP는 모든 IP에 대한 즉각적인 접근성을 제공하지만, 최종 사용자를 위한 인터페이스로는 충분하지 않다. "제어 레지스터는 오프셋 0x10에 있고 결과는 0x20에서 읽어야 한다"와 같은 저수준의 세부 정보를 사용자에게 노출하는 것은 바람직하지 않다.³⁰ 대신,

my_accelerator.run(data)와 같이 직관적이고 추상화된 고수준 API를 제공하는 것이 이상적이다. 이 장에서는 DefaultIP를 넘어 특정 하드웨어에 최적화된 커스텀 파이썬 드라이버를 제작하고, PYNQ가 이를 자동으로 바인딩하는 우아한 메커니즘에 대해 설명한다.

커스텀 드라이버 생성 과정

PYNQ에서 커스텀 드라이버를 만드는 과정은 직관적이면서도 강력하다 ²⁸:

1. 상속: 새로운 파이썬 클래스를 정의하되, `pynq.overlay.DefaultIP`를 상속받는다.
2. 생성자 구현: 생성자 `__init__` 메소드는 반드시 `description` 딕셔너리를 인자로 받아야 하며, `super().__init__(description=description)`을 호출하여 부모 클래스의 생성자에 이 딕셔너리를 전달해야 한다. 이 과정을 통해 DefaultIP의 기본 기능, 특히 내부 MMIO 객체의 생성이 올바르게 수행된다.
3. 고수준 메소드 추가: 클래스에 새로운 메소드를 추가하여 저수준 레지스터 상호작용을 캡슐화한다. 예를 들어, 두 개의 입력을 받아 덧셈을 수행하는 가속기 IP를 위한 드라이버라면, `def add(self, a, b):`와 같은 메소드를 정의할 수 있다. 이 메소드 내부에서는 `self.write()`와 `self.read()`를 사용하여 정확한 오프셋에 값을 쓰고 결과를 읽어오는 로직을 구현한다.²⁸

bindto 메커니즘: 자동 드라이버 등록

bindto는 커스텀 드라이버가 PYNQ 프레임워크에 의해 자동으로 인식되고 바인딩되도록 하는

핵심 메커니즘이다.⁹

- 커스텀 드라이버 클래스 내부에 **bindto**라는 이름의 클래스 속성(class attribute)을 정의한다.
- **bindto**는 문자열 리스트이며, 각 문자열은 이 드라이버가 처리하도록 설계된 IP의 전체 VLNV 식별자이다 (예: **bindto** = ['xilinx.com:hls:add:1.0']).²⁹
- PYNQ는 내부적으로 메타클래스(RegisterIP)를 사용하여, 파이썬 인터프리터가 커스텀 드라이버 클래스가 포함된 모듈을 임포트(import)할 때 DefaultIP의 모든 서브클래스를 자동으로 스캔한다. 그리고 각 서브클래스와 그 **bindto** 속성 정보를 전역 **_ip_drivers** 레지스트리에 등록한다.⁹

이 메커니즘의 결과로, Overlay 클래스의 **_assign_drivers** 함수가 실행될 때, HWH 파일에서 읽어온 IP의 VLNV가 레지스트리에 등록된 값과 일치하는 것을 발견한다. 그러면 DefaultIP 대신 해당 커스텀 드라이버를 IP에 할당하게 된다. 이로써 사용자는 별도의 설정 없이 자동으로 고수준의 사용하기 쉬운 API를 얻게 된다.

DefaultIP와 커스텀 드라이버 상호작용 비교

다음 표는 DefaultIP를 통한 저수준 상호작용과 커스텀 드라이버를 통한 고수준 상호작용의 차이를 명확하게 보여준다.

기능	DefaultIP를 통한 상호작용	커스텀 AddDriver를 통한 상호작용
인스턴스화	<code>add_ip = overlay.scalar_add</code>	<code>add_ip = overlay.scalar_add</code> (사용자 코드 변경 없음)
연산 수행 (3 + 4)	<code>add_ip.write(0x10, 3)</code> <code>add_ip.write(0x18, 4)</code>	<code>result = add_ip.add(3, 4)</code>
결과 읽기	<code>result = add_ip.read(0x20)</code>	(메소드에서 결과가 반환됨)
필요 지식	사용자는 IP의 레지스터 맵(오프셋 0x10, 0x18, 0x20)을 알아야 함.	사용자는 <code>add()</code> 메소드의 시그니처만 알면 됨.
코드 가독성	낮음. "매직 넘버"가 코드의 의도를 불분명하게 함.	높음. 코드가 자체적으로 문서화됨.

이 표는 커스텀 드라이버 작성이 PYNQ 오버레이 설계에서 왜 모범 사례(best practice)인지를 간결하고 설득력 있게 보여준다. 사용성과 유지보수성의 극적인 향상은 PYNQ 프로젝트의 핵심 목표와 정확히 일치한다.

bindto 메커니즘과 자동 등록 기능은 강력하고, 분리되어 있으며, 확장 가능한 드라이버 생태계를 조성한다. 이는 커뮤니티 참여를 촉진하고 코드 재사용성을 극대화한다. 예를 들어, 한 개발자가 유용한 HLS IP와 그에 상응하는 PYNQ 드라이버를 만들었다고 가정해보자. 그는 이 드라이버를 표준 파이썬 라이브러리로 패키징하여 PyPI(Python Package Index)에 게시할 수 있다 (예: `pynq-cdma`³¹). 완전히 다른 프로젝트를 진행하는 다른 사용자가 자신의 Vivado 설계에 동일한 HLS IP를 사용한다면, 그는 단순히

`pip install the-new-driver-package` 명령을 실행하기만 하면 된다. 그 후 자신의 오버레이를 PYNQ에서 로드하면, 프레임워크는 **bindto** 속성의 VLNV와 HWH 파일의 VLNV가 일치함을 감지하고 설치된 커스텀 드라이버를 자동으로 바인딩한다. 이는 커뮤니티가 하드웨어-소프트웨어 "라이브러리"를 만들어 기여하고, 이것이 어떤 PYNQ 프로젝트에도 매끄럽게 통합될 수 있는 확장 가능한 생태계를 구축한다. 이는 PYNQ 오버레이가 소프트웨어 라이브러리의 하드웨어 등가물이라는 비전을 실현하는 것이다.³

제 6장: 종합 - 전체 프로세스 сквозной 워크스루

이전 장들에서 논의된 개념들을 통합하여, 간단한 AXI-Lite IP가 Vivado에서 생성되어 주피터 노트북에서 제어되기까지의 전 과정을 단계별로 추적해 본다. 이 워크스루는 먼저 DefaultIP를 사용한 초기 상호작용을 보여주고, 이어서 커스텀 드라이버를 통한 고수준 제어로 전환하는 과정을 시연한다.

1. **Vivado** 설계: Zynq PS가 "AXI Multiplier"라는 간단한 커스텀 IP에 AXI-Lite 인터페이스로 연결된 블록 다이어그램을 가정한다. Vivado의 주소 편집기(Address Editor)를 통해 이 IP에는 0x43C00000이라는 기준 주소가 할당된다.
2. **HWH** 파일 생성: 비트스트림 생성 후 만들어진 .hwh 파일에는 이 IP에 대한 XML 항목이 포함된다. 이 항목에는 모듈 이름(axi_multiplier_0), VLNV, 그리고 기준 주소를 명시하는 <MEMMAP> 태그가 포함될 것이다.
3. **PYNQ - 1단계 (기본 상호작용):**
 - 주피터 노트북 셀에서 오버레이를 로드한다:

```
Python
from pynq import Overlay
ol = Overlay('multiplier.xsa')
```
 - `ol?` (또는 `help(ol)`) 명령을 사용하여 오버레이의 내용을 확인하면, `axi_multiplier_0` 속성이 `pynq.overlay.DefaultIP` 타입으로 생성되었음을 알 수 있다.

- DefaultIP가 제공하는 저수준 **read/write** 메소드를 사용하여 IP를 직접 제어한다. 이는 하드웨어의 기능성을 즉시 검증하는 데 매우 유용하다.²⁸

Python

```
mult_ip = ol.axi_multiplier_0
```

```
# 입력 레지스터(오프셋 0x10, 0x18)에 값 쓰기
```

```
mult_ip.write(0x10, 7)
```

```
mult_ip.write(0x18, 6)
```

```
# 결과 레지스터(오프셋 0x20)에서 값 읽기
```

```
result = mult_ip.read(0x20)
```

```
print(result)
```

```
# 출력: 42
```

4. PYNQ - 2단계 (커스텀 드라이버):

- 다음으로, 더 직관적인 API를 제공하기 위해 커스텀 드라이버를 정의한다.

Python

```
from pynq import DefaultIP
```

```
class MultiplierDriver(DefaultIP):
```

```
    def __init__(self, description):
```

```
        super().__init__(description=description)
```

```
    bindto = ['xilinx.com:hls:axi_multiplier:1.0'] # IP의 VLNv와 일치해야 함
```

```
    def multiply(self, a, b):
```

```
        self.write(0x10, a)
```

```
        self.write(0x18, b)
```

```
        return self.read(0x20)
```

- 이 드라이버 클래스를 정의한 후, 오버레이를 다시 로드하여 PYNQ가 새로운 드라이버를 인식하고 등록하도록 한다.

Python

```
ol = Overlay('multiplier.xsa')
```

- 다시 ol? 명령으로 확인하면, 이제 axi_multiplier_0 속성이 __main__.MultiplierDriver 타입(또는 드라이버가 정의된 모듈의 경로)으로 바인딩된 것을 볼 수 있다.
- 이제 최종적으로 추상화된 고수준 API를 사용하여 IP를 제어한다.

Python

```
mult_ip = ol.axi_multiplier_0
```

```
result = mult_ip.multiply(7, 6)
```

```
print(result)
```

```
# 출력: 42
```

이 워크스루는 저수준 하드웨어 설계에서 시작하여 고수준의 추상화된 파이썬 제어에 이르는 매끄러운 PYNQ 워크플로우를 구체적으로 보여준다. 이는 PYNQ가 어떻게 복잡성을 단계적으로 추상화하고 개발자에게 높은 생산성을 제공하는지에 대한 실질적인 증명이다.

결론

PYNQ 프레임워크의 힘은 저수준 하드웨어 기술서를 고수준의 상호작용적인 파이썬 객체로 변환하는 과정을 자동화하는 정교하고 다층적인 소프트웨어 아키텍처에서 비롯된다. 본 보고서에서 분석한 바와 같이, 이 변환 과정은 여러 핵심 메커니즘의 유기적인 결합을 통해 이루어진다.

첫째, 메타데이터 기반의 하드웨어 발견 메커니즘이다. Vivado가 생성하는 XSA/HWH 파일은 단순한 핸드오프 파일을 넘어, PYNQ가 런타임에 하드웨어 토폴로지를 동적으로 파악할 수 있게 하는 기계 판독 가능한 청사진 역할을 한다. PYNQ-Metadata 라이브러리는 이 청사진을 해석하여 프레임워크가 사용할 수 있는 구조화된 인메모리 모델로 변환한다.

둘째, 커널 수준 접근의 추상화이다. `pynq.mmio.MMIO` 클래스는 리눅스의 `/dev/mem`과 `mmap` 시스템 콜을 활용하여 사용자 공간 파이썬 프로세스와 물리 메모리 간의 다리를 놓는다. 이 과정에서 페이지 정렬, 루트 권한, 메모리 보호와 같은 복잡한 운영 체제 수준의 문제들을 캡슐화하여 개발자에게는 간단한 `read/write` API만을 노출한다.

셋째, 동적이고 확장 가능한 드라이버 모델이다. PYNQ는 파싱된 모든 IP에 대해 `DefaultIP`라는 범용 드라이버를 즉시 제공하여 신속한 프로토타이핑을 가능하게 한다. 더 나아가, `DefaultIP`를 상속하고 `bindto` 속성을 사용하는 간단한 규칙을 통해 개발자는 특정 IP에 최적화된 고수준 드라이버를 손쉽게 제작하고, PYNQ는 이를 자동으로 바인딩한다. 이 메커니즘은 코드 재사용성을 높이고 커뮤니티 기반의 하드웨어-소프트웨어 라이브러리 생태계를 촉진한다.

결론적으로, PYNQ는 하드웨어와 소프트웨어 사이의 간극을 메우기 위해 설계된 지능적인 추상화 계층의 집합체이다. 이 아키텍처는 FPGA 개발의 진입 장벽을 낮추고, 소프트웨어 개발자들이 익숙한 방식으로 하드웨어 가속의 이점을 활용할 수 있도록 함으로써, 진정한 의미의 하드웨어-소프트웨어 공동 설계를 높은 생산성의 환경에서 실현시킨다.

참고 자료

1. PYNQ Framework on TityraCore Zynq® 7000 | Numato Lab Help Center, 9월 24, 2025에 액세스, <https://numato.com/blog/pynq-framework-on-tityracore-zynq-7000/>
2. PYNQ | Python Productivity for AMD Adaptive Computing platforms, 9월 24, 2025에 액세스, <http://www.pynq.io/>

3. PYNQ Overlays — Python productivity for Zynq (Pynq) - Read the Docs, 9월 24, 2025에 액세스, https://pynq.readthedocs.io/en/latest/pynq_overlays.html
4. AXI GPIO, Memory-Mapped I/O MMIO, Read/Write Using C Pointer ..., 9월 24, 2025에 액세스, <https://www.hackster.io/fpgaps/axi-gpio-memory-mapped-i-o-mmio-read-write-using-c-pointer-0db8a9>
5. Xilinx Wiki - Confluence, 9월 24, 2025에 액세스, <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841693/HSI+debugging+and+optimization+techniques?f=print>
6. Overlay Design — Python productivity for Zynq (Pynq), 9월 24, 2025에 액세스, https://pynq.readthedocs.io/en/latest/overlay_design_methodology/overlay_design.html
7. Completed Developing Zynq UltraScale+ MPSoC Software with Xilinx SDK Lab 1, Lab 2, Lab 3 and Lab 4 - element14 Community, 9월 24, 2025에 액세스, <https://community.element14.com/challenges-projects/design-challenges/path2programmable/b/blog/posts/completed-developing-zynq-ultrascale-mpsoc-software-with-xilinx-sdk-lab-1-lab-2-lab-3-and-lab-4>
8. Source code for pynq.overlay - Read the Docs, 9월 24, 2025에 액세스, https://pynq.readthedocs.io/en/v2.3/_modules/pynq/overlay.html
9. pynq.overlay Module — Python productivity for Zynq (Pynq) v1.0 - Read the Docs, 9월 24, 2025에 액세스, https://pynq.readthedocs.io/en/v2.1/pynq_package/pynq.overlay.html
10. .bit file to .hwh file - Adaptive Support, 9월 24, 2025에 액세스, https://adaptivesupport.amd.com/s/question/0D52E00006hpSAISA2/bit-file-to-hwh-file?language=en_US
11. Creating a new Vitis project - Real Digital, 9월 24, 2025에 액세스, <https://www.realdigital.org/doc/1ea17599778722bbec279c5d2a61e9a8>
12. xilinx-wiki.atlassian.net, 9월 24, 2025에 액세스, [https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841693/HSI+debugging+and+optimization+techniques?f=print#:~:text=The%20XSA%20\(Xilinx%20Support%20Archive,a%20Block%20Design%20\(BD\).](https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841693/HSI+debugging+and+optimization+techniques?f=print#:~:text=The%20XSA%20(Xilinx%20Support%20Archive,a%20Block%20Design%20(BD).)
13. pynq.pl_server.embedded_device Module - Read the Docs, 9월 24, 2025에 액세스, https://pynq.readthedocs.io/en/latest/pynq_package/pynq.pl_server/pynq.pl_server.embedded_device.html
14. Can the pynq.overlay class load an XSA file instead of a BIT file? - Support, 9월 24, 2025에 액세스, <https://discuss.pynq.io/t/can-the-pynq-overlay-class-load-an-xsa-file-instead-of-a-bit-file/5500>
15. My XSA won't load, how do I load my overlay through the hwh and bit method instead?, 9월 24, 2025에 액세스, <https://discuss.pynq.io/t/my-xsa-wont-load-how-do-i-load-my-overlay-through-the-hwh-and-bit-method-instead/5575/2>
16. Xilinx/PYNQ-Metadata: PYNQ-Metadata provides an ... - GitHub, 9월 24, 2025에 액세스, <https://github.com/Xilinx/PYNQ-Metadata>

17. pynq.metadata-runtime_metadata_parser Module, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/latest/pynq_package/pynq.metadata/pynq.metadata.runtime_metadata_parser.html
18. Customizing PYNQ for Pynq-Z1 - Support, 9월 24, 2025에 액세스,
<https://discuss.pynq.io/t/customizing-pynq-for-pynq-z1/1882>
19. python/pynq/mmio.py · v1.2 · paulrr2 / PYNQ - GitLab at Illinois, 9월 24, 2025에 액세스,
<https://gitlab-03.engr.illinois.edu/paulrr2/PYNQ/-/blob/v1.2/python/pynq/mmio.py>
20. pynq.mmio — Python productivity for Zynq (Pynq) v1.0, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.1/_modules/pynq/mmio.html
21. Using PYNQ MMIO (Memory Mapped IO) - YouTube, 9월 24, 2025에 액세스,
<https://www.youtube.com/watch?v=zbumlTJQ2Z8>
22. MMIO — Python productivity for Zynq (Pynq) v1.0 - Read the Docs, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.1/pynq_libraries/mmio.html
23. pynq.mmio Module — Python productivity for Zynq (Pynq) v1.0, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.3/pynq_package/pynq.mmio.html
24. pynq.mmio — Python productivity for Zynq (Pynq) - Read the Docs, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.5.1/_modules/pynq/mmio.html
25. Overlay — Python productivity for Zynq (Pynq) - Read the Docs, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.6.1/pynq_libraries/overlay.html
26. Loading an Overlay — Python productivity for Zynq (Pynq) - Read the Docs, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.6.1/pynq_overlays/loading_an_overlay.html
27. pynq.overlay — Python productivity for Zynq (Pynq) v1.0, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.4/_modules/pynq/overlay.html
28. Overlay Tutorial — Python productivity for Zynq (Pynq), 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/latest/overlay_design_methodology/overlay_tutorial.html
29. Overlay Tutorial — Python productivity for Zynq (Pynq) v1.0 - Read the Docs, 9월 24, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.4/overlay_design_methodology/overlay_tutorial.html
30. Clarification on Driver Creation and custom overlay - Support - PYNQ, 9월 24, 2025에 액세스,
<https://discuss.pynq.io/t/clarification-on-driver-creation-and-custom-overlay/5178>
31. pynq-cdma - PyPI, 9월 24, 2025에 액세스,
<https://pypi.org/project/pynq-cdma/>
32. PYNQ AXI GPIO and Memory Mapped I/O (MMIO) Example: Control Blinking LEDs by DIP Switch - YouTube, 9월 24, 2025에 액세스,
<https://m.youtube.com/watch?v=xw1JPAHh1qI>
33. PYNQ AXI GPIO & MMIO: Control Blinking LEDs With DIP Switch - Hackster.io, 9월 24, 2025에 액세스,
<https://www.hackster.io/fpgaps/pynq-axi-gpio-mmio-control-blinking-leds-with-dip-switch-23cd50>